Authored by: Arpit Raj, LNMIIT Jaipur
Date: Sunday, October 25, 2026
© 2026 Arpit Raj. All rights reserved.

Advanced C++

Override, Virtual Destructors & Abstract Classes

The override Keyword

let `A` have `virtual show()` and let `B` have `show(int x) override`.
Since parameter list differs -> signature differs which means override isnt possible.

`override` keyword basically verifies if an intended override attempt was correct or not.

- **without override:** legal declaration, only hides the base name.
- **with override:** mistake caught immediately.

⚠️ INVALID OVERRIDES (SIGNATURE MISMATCH)

- # `virtual void show() const` AND `void show()` have different signature so using `override` gives compile error.
- # `virtual int fun()` AND `double fun() override` is also invalid because overriding needs compatible return type.

Virtual vs Non-Virtual Destructors

I. Non Virtual

```
class A { public: ~A() { }
};

class B : public A { public: ~B() { }
};
```

now if we write `A* ptr = new B();` and `delete ptr;`

> 🚨 **CRITICAL MEMORY WARNING**
>
> C++ standard states that deleting a derived object through a base pointer of non virtual destructor type gives undefined behaviour.

S1) `new B()` allocates memory for full B (own + A inherit)

S2) `ptr` points to A part of B object

S3) since not virtual uses static binding and deletes only A part

- S4) i.e only A's destructor is called.

now the entire memory block is released. B's destructor is skipped, so if B had its own allocated memory it results into memory leakage.

II. virtual destructor

so the solution is to use `virtual ~A()` and `~B() override`.

now if we call `delete ptr`, it uses dynamic binding, instead of blindly calling `~A()`, it looks at hidden vptr pointing to B's vtable.
the vtable directs to call most derived destructor first.
and in C++, a derived class destructor automatically calls its base class destructor before finish.

O/p -> B destructor, A destructor.

Pure Virtual Functions & Abstract Classes

Purely virtual function (formal definition later, this is just a brief)
every shape has area, but formula differs (circle = πr^2, rectangle = $l \times w$).
Instead of a default implementation, we declare `area()` a purely virtual.

```
class Shape {
public: virtual double area() = 0;
};
```

`= 0` syntax makes a function pure virtual. it tells Shape it's an abstract class and cant be instantiated directly.

> **Abstract Class**
>
> a class is abstract if it has atleast one pure virtual function that hasn't been overriden to provide an implementation.

> **FORMAL definition pure virtual function**
>
> a function in a base class that has no implementation in its base class, and forces because the concept is too abstract at that level. It forces derived class to provide their own implementations.

implementing pure virtual function

```
class Shape {
public: virtual double area() = 0;
};

class rectangle : public Shape {
public:
    int length, width; // corrected declaration syntax from notes
    rectangle(int l, int w) {
        this->length = l;
        this->width = w;
    }
    double area() override {
        return length * width;
    }
};
```

- # `Shape s;` will give error because implementation is abstract at that level.
- # This concept is useful because it provides a common interface to execute appropriate derived implementation.

Interfaces & Contracts

when to use interface like class
interface is a design concept we use when we dont want base class to decide implementation, we only want derived class to decide and use that implementation.

Eg

```
class Pay {
    virtual void pay(int amount) = 0;
    virtual ~Pay() { }
};

class UPI : public Pay {
    void pay(int amount) override { }
};

main() {
    UPI upi;
    Pay * ptr;
    ptr = &upi;
    ptr->pay(500);
}
```

(Note: Pay defines the contract, UPI provides the implementation)

Abstract class is basically a contract plus a shared functionality
imagine Pay also had a public function to show transaction count.
so now derived class must implement the pay function but they can reuse show. This is abstract class.